

Invariant Pretraining for Robust Code Representations

Yifeng He, Yundi Xu, Christopher Castro Gaw Gonzalo, Zili Wang, Hao Chen

University of California, Davis The University of Hong Kong

LMPL 2026 Oakland, CA, USA

Small Encoders Still Do the Code-to-Code Work

Where discriminative code models sit today

- ▶ Deduplicating training corpora
- ▶ Retrieving and ranking code
- ▶ Flagging near-duplicates in agent workflows

Why encoders, not LLMs

CodeBERT-class encoders have about 125M parameters, yet match or beat 7B–15B generation models on clone detection and classification, with less storage and lower inference cost.

So their failure modes still matter

These models run on every snippet in a corpus or every candidate in a retrieval pool. A systematic blind spot is paid at scale.

Question for this talk:

When only the syntax changes, do these encoders still treat it as the *same program*?

Invariant Programs: Same Behavior, Different Syntax

Original

```
int count(int[] a, int t) {  
    int n = 0;  
    for (int i = 0; i < a.length; i++) {  
        if (a[i] == t) n++;  
    }  
    return n;  
}
```



Invariant

```
int count(int[] qx, int zr) {  
    int vk = 0;  
    { int ub = 0;  
        while (ub < qx.length) {  
            if (qx[ub] != zr) {} else { vk = vk + 1; }  
            ub = ub + 1;  
        }  
    }  
    return vk;  
}
```

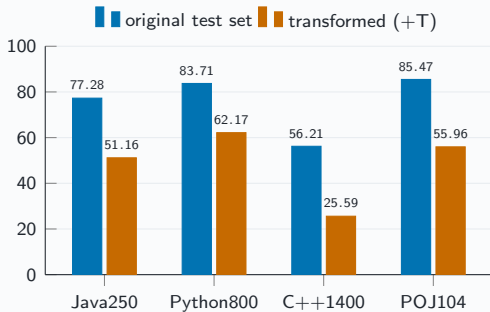
Applied: VarRe, F2W, PP2AA then AA2EA, RevIf. Every rewrite is one a developer might write by style or convention.

What a robust encoder should do

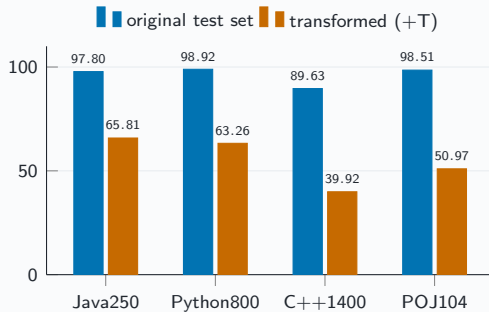
Embed both programs close together, so a clone detector retrieves one from the other and a classifier assigns them the same label.

The Gap: Equivalent Rewrites Break Encoder Representations

Clone detection (MAP@R), CodeBERT



Code classification (accuracy), CodeBERT



- ▶ Same programs, same labels, same behavior: clone detection loses 21.5–30.6 points; classification loses 32–50 points.
- ▶ Not a CodeBERT quirk: all four encoder baselines degrade, and so do newer reference encoders (CodeSage 1.3B: 72.43 → 25.92 on Java250).

What Prior Work Leaves Open

Method	Training signal	Equivalent variants in a batch	Stage
SPAT	code only	no contrastive objective	fine-tuning
Corder, ContraCode	code only	negatives (InfoNCE)	pretraining from scratch
ContraBERT	paired NL-PL	negatives (MoCo, InfoNCE)	continued pretraining
InvPT (ours)	code only	positives (SupCon mask)	continued pretraining

What is missing

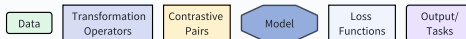
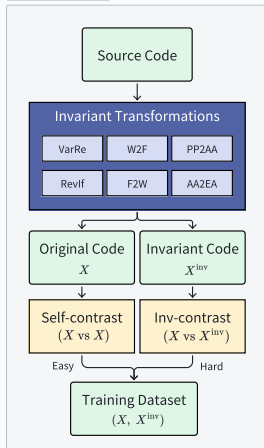
No systematic measurement of how far current encoders degrade under semantics-preserving transformations, and no code-only pretraining method that keeps equivalent variants together instead of pushing them apart.

What we contribute

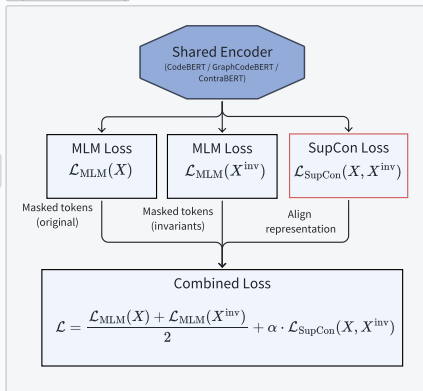
- ▶ A **measurement** of the gap across 4 encoders, 2 tasks, 4 datasets.
- ▶ **InvPT**, a code-only continued-pretraining method that treats every equivalent variant as a positive.

InvPT in One Picture

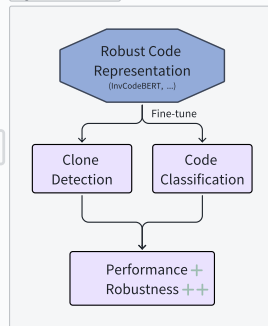
Stage 1: Data Preparation



Stage 2: Invariant Pre-Training



Stage 3: Downstream Tasks



- 1 Rewrite each function, pair it with itself (easy) and its invariant (hard) → 2 Continue pretraining with MLM + multi-positive SupCon → 3 Fine-tune as usual

Six AST-Level Operators with Checked Preconditions

Operator	Category	Transformation rule	Lang.
VarRe	Syntax	Replace all variable names with random valid identifiers	🔥 C/C++ 🧩
W2F	Syntax	$\text{while}(p) \{B\} \Rightarrow \text{for}(; p;) \{B\}$	🔥 C/C++
F2W	Syntax	$\text{for}(init; p; inc) \{B\} \Rightarrow \text{init}; \text{while}(p) \{B; inc\}$	🔥 C/C++
PP2AA	Syntax	$x++ \Rightarrow x += 1$	🔥 C/C++
AA2EA	Syntax	$x += 1 \Rightarrow x = x + 1$	🔥 C/C++ 🧩
RevIf	Branching	$\text{if}(p) \{B_1\} \text{ else } \{B_2\} \Rightarrow \text{if}(\neg p) \{B_2\} \text{ else } \{B_1\}$	🔥 C/C++ 🧩

Equivalence by construction

- ▶ F2W only when the body has no `continue`
- ▶ PP2AA only when `x++`'s value is unused, on primitive numeric types
- ▶ VarRe is a consistent, injective rename per scope

Honest caveat

Rewrites are implemented on the AST (Java operators from SPAT, clang, Python ast), not formally verified. AA2EA on Python is a heuristic: `x += e` and `x = x + e` differ for aliased lists.

InfoNCE Pushes Equivalent Programs Apart

The false-negative problem

We apply several operators per function, so two variants of the same function often land in one batch. InfoNCE has one positive per anchor and treats the other variant as a **negative**: it pushes them apart, against the invariance we want.

Fix: a multi-positive SupCon mask

Key every sample by a hash of its *canonical* source:

$$\text{id}(c) = \text{SHA-256}(c)[:8], \quad M_{ij} = \mathbb{K}[\text{id}_i = \text{id}_j \wedge i \neq j]$$

All augmentations of a function are positives.

Deterministic, no coordination across workers, and it reduces to InfoNCE when no collisions occur.

Batch of $B = 4$ pairs from sources $[f, g, h, f]$

	0	1	2	3	4	5	6	7
0 (f)	-			•	•			•
1 (g)		-				•		
2 (h)			-				•	
3 (f)	•			-	•			•
4 (f)	•			•	-			•
5 (g)		•				-		
6 (h)			•				-	
7 (f)	•			•	•			-

Rows 0–3 original, 4–7 augmented. • = positive.
Each f anchor has 3 positives; g and h have 1.

Mixed-Difficulty Positives in One Training Set

Self-contrast: easy

Same code X twice, different 15% MLM masks. The pair differs only in masked positions (the SimCSE idea, new to code pretraining).

Invariant contrast: hard

X against X^{inv} . After VarRe and F2W, an anchor and its positive may share almost no tokens, so the model must look past surface form.

$$\mathcal{L}(X, X^{\text{inv}}) = \frac{\mathcal{L}_{\text{MLM}}(X) + \mathcal{L}_{\text{MLM}}(X^{\text{inv}})}{2} + \alpha \mathcal{L}_{\text{SupCon}}$$

- ▶ Both pair types go into **one training set** before shuffling, not staged: a curriculum would shift the positive distribution mid-training.
- ▶ A single shared encoder, no momentum encoder: our positives are equivalent by construction, so the gradient signal is unambiguous.

Setup: Four Encoders, Two Tasks, Four Datasets

Each released encoder gets an InvPT twin

Baseline	CodeBERT	GraphCodeBERT	ContraBERT_C	ContraBERT_G
+ InvPT	InvCodeBERT	InvGraphCodeBERT	InvContraBERT_C	InvContraBERT_G

Continued pretraining

- ▶ CodeSearchNet **Java and Python only**
- ▶ 3 epochs, about 80 H100 GPU-hours per model

The +T test sets

All six operators applied cumulatively. Training saw single operators only, so the compositions are unseen, but the **operator family is the same**.

Downstream

- ▶ Clone detection (MAP@R) and classification (accuracy)
- ▶ POJ104 (C/C++), CodeNet Java250, Python800, C++1400
- ▶ **C/C++ never appears in InvPT pretraining**

Standard Performance Is Preserved or Improved

Model	Java250	Python800	C++1400	POJ104
CodeBERT	77.28	83.71	56.21	85.47
GraphCodeBERT	82.15	86.06	58.76	84.09
ContraBERT_C	82.19	85.60	57.65	89.58
ContraBERT_G	83.47	85.97	59.15	90.80
CodeSage	72.43	61.11	51.53	78.20
ModernBERT	80.51	83.16	59.05	88.22
InvCodeBERT	82.44 (+6.7%)	85.63 (+2.3%)	57.80 (+2.8%)	88.81 (+3.9%)
InvGraphCodeBERT	84.41 (+2.8%)	86.73 (+0.8%)	59.23 (+0.8%)	90.66 (+7.8%)
InvContraBERT_C	82.63 (+0.5%)	86.16 (+0.7%)	58.09 (+0.8%)	89.28 (-0.3%)
InvContraBERT_G	84.53 (+1.3%)	86.34 (+0.4%)	59.23 (+0.1%)	91.25 (+0.5%)

- ▶ Clone detection on the original test sets (MAP@R): InvPT improves each baseline family on most benchmarks, up to **+7.8%** (InvGraphCodeBERT on POJ104).
- ▶ Classification is near saturation (97.8–99.1% on three datasets); InvPT stays within -0.1 to 0.4% of its baseline.
- ▶ Robustness is **not bought** with standard accuracy.

Robustness: Every InvPT Model Beats Its Baseline on +T

Model	Java250	Python800	C++1400	POJ104
CodeBERT	51.16	62.17	25.59	55.96
GraphCodeBERT	53.52	65.89	30.26	58.63
ContraBERT_C	49.64	63.41	27.09	62.76
ContraBERT_G	53.79	65.98	29.11	64.20
CodeSage	25.92	25.89	21.33	52.87
ModernBERT	41.06	52.85	24.66	52.86
InvCodeBERT	59.27 (+15.9%)	70.55 (+13.5%)	31.43 (+22.8%)	62.03 (+10.8%)
InvGraphCodeBERT	62.30 (+16.4%)	73.91 (+12.2%)	33.68 (+11.3%)	67.32 (+14.8%)
InvContraBERT_C	60.68 (+22.2%)	74.34 (+17.2%)	32.07 (+18.4%)	63.53 (+1.2%)
InvContraBERT_G	62.24 (+15.7%)	74.60 (+13.1%)	33.10 (+13.7%)	68.63 (+6.9%)

- ▶ Clone detection on transformed test sets: gains in all 16 model–dataset pairs, **+1.2% to +22.8%** relative.
- ▶ InvGraphCodeBERT beats both ContraBERT variants everywhere; InvCodeBERT still trails them on POJ104 (62.03 vs. 62.76 and 64.20).
- ▶ The newer reference encoders (CodeSage, ModernBERT) are the two least robust rows on every dataset.

How Much of the Gap Does InvPT Close?

Clone detection +T (pp vs. baseline)

Model	Java250	Python800	C++1400	POJ104
InvCodeBERT	+8.11	+8.38	+5.84	+6.07
InvGraphCodeBERT	+8.78	+8.02	+3.42	+8.69
InvContraBERT_C	+11.04	+10.93	+4.98	+0.77
InvContraBERT_G	+8.45	+8.62	+3.99	+4.43

Consistent gains

Clone detection: median **+8.07 pp**, up to +11.04.
Classification: median **+3.56 pp**, skewed; +19.16 is an outlier (next is +10.67).

Classification +T (pp vs. baseline)

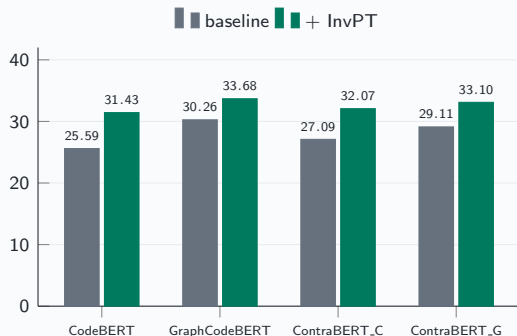
Model	Java250	Python800	C++1400	POJ104
InvCodeBERT	+4.29	+19.16	+3.77	+2.70
InvGraphCodeBERT	+3.79	+6.08	+2.71	+0.63
InvContraBERT_C	+4.02	+10.67	+1.05	+2.05
InvContraBERT_G	+2.91	+4.83	+1.65	+3.36

But most of the gap remains

InvPT recovers a median **29.6%** of the orig→+T drop on clone detection and **8.7%** on classification. The part we close is smaller than the part left open.

Invariance Learned on Java and Python Transfers to C/C++

C++1400 clone detection +T (MAP@R)



C/C++ is held out of InvPT pretraining

No C/C++ program is transformed during continued pretraining, yet C++1400 gains **+3.42** to **+5.84 pp** on clone detection and **+1.05** to **+3.77 pp** on classification.

Why we think it transfers

- ▶ The operators target language-agnostic structure: loops, branches, increments, names.
- ▶ Java, Python, and C/C++ share keywords and operators the sub-word tokenizer encodes alike.

Not a claim of cross-lingual alignment, which would need parallel data.

Ablation: Multi-Positive Contrast Drives the Gains

Clone detection +T, pp vs. full InvCodeBERT

Variant	Java250	Python800	C++1400	POJ104
w/o contrastive loss	-3.15	-2.28	-2.96	-4.52
w/o self-contrast	-1.35	-0.28	-1.10	+0.39
InvCodeBERT + NL	+2.08	-0.68	-0.42	+2.08

CodeBERT-based variants; full orig/+T numbers are in the paper's ablation table.

Paired natural language is not needed

+NL moves +T by 0.76 and orig by -0.18 on average, with mixed signs across datasets. Not worth a bi-modal corpus.

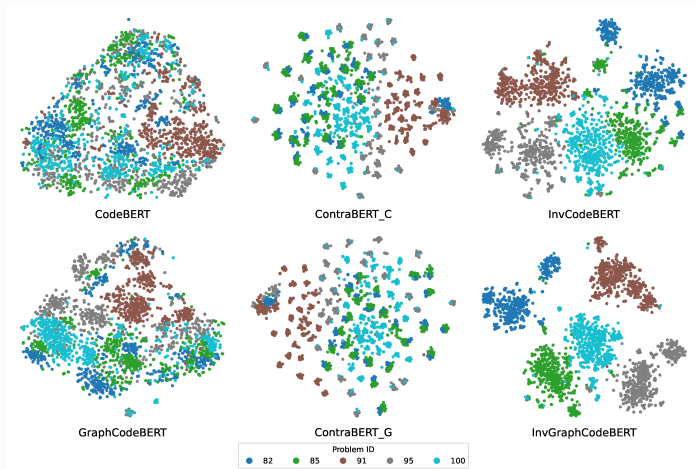
Contrastive loss is the main driver

MLM on transformed code alone costs 2.28–4.52 pp. On original POJ104 it even falls below CodeBERT (83.87 vs. 85.47): exposure without alignment is not enough.

Self-contrast adds a small, stable gain

Removing it lowers every original benchmark by 0.5–1.1 points.

What the Embedding Space Looks Like



t-SNE of five POJ104 problems, 500 solutions each, with invariant transformations applied.

- ▶ **Base models:** heavy overlap; rewritten programs sit near unrelated solutions.
- ▶ **ContraBERT:** local groupings, still mixed.
- ▶ **InvPT:** tight, separated clusters per problem.

Qualitative only; consistent with the tables, not evidence on its own.

Three Things to Remember

- 1 The four encoder baselines lose 20–50 points on semantically equivalent rewrites. Surface form still drives their embeddings.
- 2 InvPT: code alone supplies the signal. Treat every equivalent variant as a positive, mix easy and hard pairs, and continued pretraining gains a median +8.07 pp (clone) and +3.56 pp (classification), while preserving standard accuracy and transferring to unseen C/C++.
- 3 InvPT closes part of the gap, not most of it. What remains is the open problem.

Next: from invariance to equivariance

Generation should not ignore a rename; it should rename consistently. For a transformation T and model g : $g(T(x)) = M_T(g(x))$, with invariance the special case $M_T = \text{id}$.

Do Encoders Treat a Rewrite as the Same Program?

```
int count(int[] qx, int zr) {  
    int vk = 0;  
    { int ub = 0;  
        while (ub < qx.length) {  
            if (qx[ub] != zr) {} else { vk = vk + 1; }  
            ub = ub + 1;  
        }  
    }  
    return vk;  
}
```

Questions

Today, only partly.

Code alone can teach them more.

yfhe@ucdavis.edu



Paper



Code